
DENNIS GUNAWAN



UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

IF351 DATABASE SYSTEMS

08 TRANSACTION PROCESSING, CONCURRENCY, AND RECOVERY TECHNIQUES



REVIEW:SELECT STATEMENT 3

- JOIN
- Set Operator

COURSE SUB LEARNING OUTCOMES (SUB-CLO)

- Sub-CLO 8
 - Students are able to explain concurrency and recovery techniques (C2)
- Sub-CLO 9
 - Students are able to use Transaction Control Language (TCL) for transaction processing (C3)

OUTLINE

- Transactions
- Concurrency
- Recovery Techniques
- Transaction Processing in MySQL

SINGLE-USER VS MULTIUSER SYSTEMS

- A DBMS is **single-user** if at most one user at a time can use the system
- It is **multiuser** if many users can use the system and hence access the database concurrently
 - For example, an airline reservations system is used by hundreds of travel agents and reservation clerks concurrently

TRANSACTIONS

An executing program that forms
a logical unit of database processing

- A transaction includes one or more database access operations (insertion, deletion, modification, or retrieval operations)
- If the database operations in a transaction do not update the database but only retrieve data, the transaction is called a **read-only transaction**
- Otherwise it is known as a **read-write transaction**

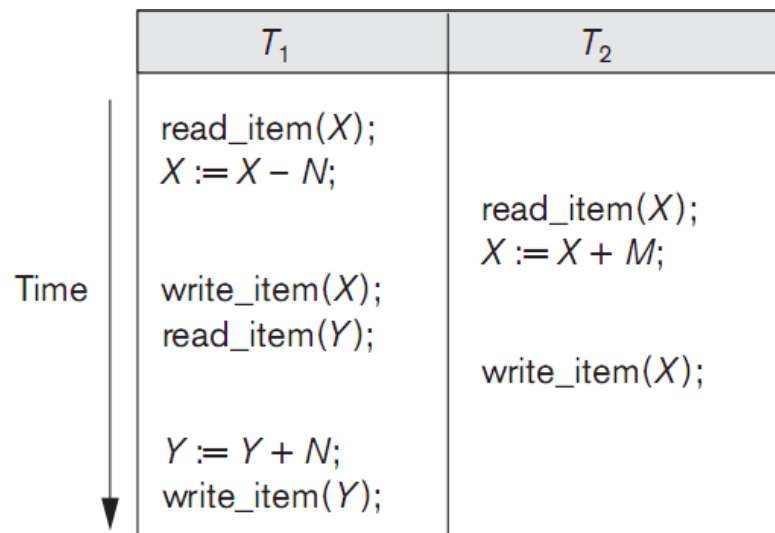
TRANSACTIONS

- The basic database access operations that a transaction can include
 - **read_item(X)**
reads a database item named X into a program variable
 - **write_item(X)**
writes the value of program variable X into the database item named X

T_1
read_item(X); $X := X - N$; write_item(X); read_item(Y); $Y := Y + N$; write_item(Y);

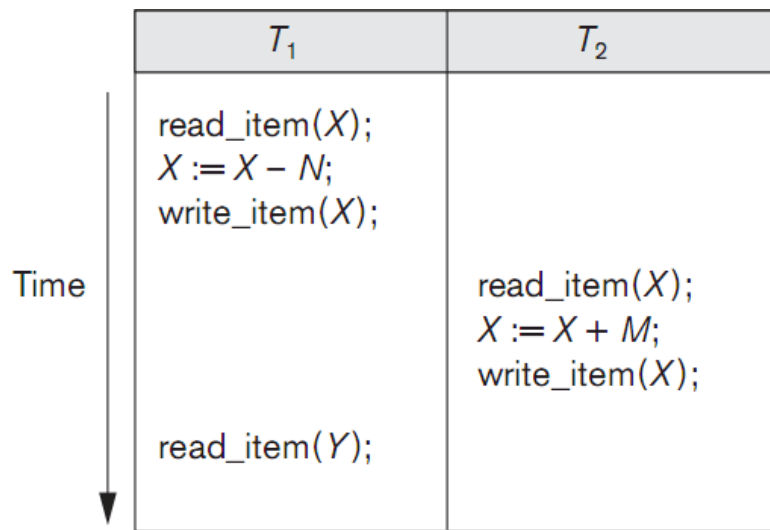
T_2
read_item(X); $X := X + M$; write_item(X);

WHY CONCURRENCY CONTROL IS NEEDED: THE LOST UPDATE PROBLEM



- This problem occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database items incorrect
- Item X has an incorrect value because its update by T_1 is lost (overwritten)

WHY CONCURRENCY CONTROL IS NEEDED: THE TEMPORARY UPDATE (OR DIRTY READ) PROBLEM



- This problem occurs when one transaction updates a database item and then the transaction fails for some reason
- Meanwhile, the updated item is accessed (read) by another transaction before it is changed back to its original value
- Transaction T_1 fails and must change the value of X back to its old value; meanwhile T_2 has read the temporary incorrect value of X

WHY CONCURRENCY CONTROL IS NEEDED: THE INCORRECT SUMMARY PROBLEM

T_1	T_3
<pre>read_item(X); X := X - N; write_item(X); read_item(Y); Y := Y + N; write_item(Y);</pre>	<pre>sum := 0; read_item(A); sum := sum + A; ⋮ read_item(X); sum := sum + X; read_item(Y); sum := sum + Y;</pre>

- If one transaction is calculating an aggregate summary function on a number of database items while other transactions are updating some of these items, the aggregate function may calculate some values before they are updated and others after they are updated
- T_3 reads X after N is subtracted and reads Y before N is added; a wrong summary is the result (off by N)

WHY RECOVERY IS NEEDED

- **A computer failure (system crash)**
 - Hardware, software, or network error occurs in the computer system during transaction execution
 - Hardware crashes are usually media failures, for example main memory failure
- **A transaction or system error**
 - Some operation in the transaction may cause it to fail
 - Integer overflow
 - Division by zero
 - Erroneous parameter values
 - Logical programming error

WHY RECOVERY IS NEEDED

- **Local errors or exception conditions detected by the transaction**
 - During transaction execution, certain conditions may occur that necessitate cancellation of the transaction
 - Data for the transaction may not be found
 - Insufficient account balance in a banking database
- **Concurrency control enforcement**
 - The concurrency control method may decide to abort a transaction because it violates serializability, or it may abort one or more transactions to resolve a state of deadlock among several transactions

WHY RECOVERY IS NEEDED

- **Disk failure**
 - Some disk blocks may lose their data because of a read /write malfunction or because of a disk read /write head crash
- **Physical problems and catastrophes**
 - Power or air-conditioning failure
 - Fire
 - Theft
 - Sabotage
 - Overwriting disks or tapes by mistake
 - Mounting of a wrong tape by the operator

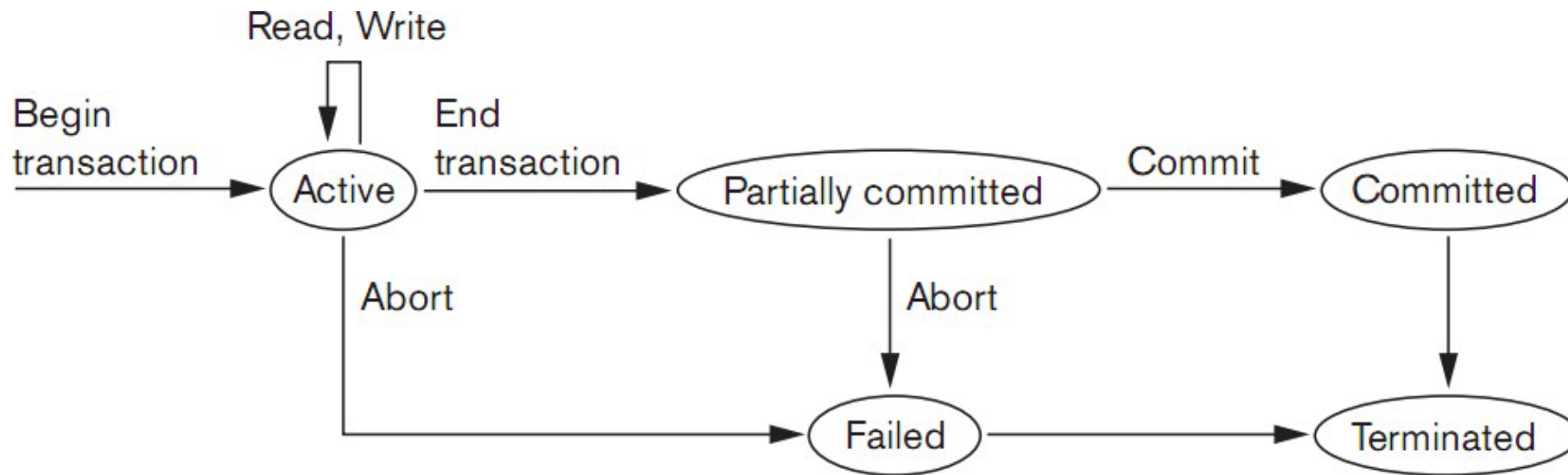
TRANSACTION STATES & OPERATIONS

A transaction is an atomic unit of work that is either completed in its entirety or not done at all

- Recovery manager keeps track of the following operations
 - BEGIN_TRANSACTION
 - READ OR WRITE
 - END_TRANSACTION
 - COMMIT_TRANSACTION
 - ROLLBACK (or ABORT)

TRANSACTION STATES & OPERATIONS

- State transition diagram illustrating the states for transaction execution



COMMIT POINT OF A TRANSACTION

- A transaction T reaches its commit point when all its operations that access the database have been executed successfully and the effect of all the transaction operations on the database have been recorded in the log
- Beyond the commit point, the transaction is said to be **committed**, and its effect is assumed to be **permanently recorded** in the database
- The transaction then writes a commit record into the log

COMMIT POINT OF A TRANSACTION

- If a system failure occurs, we search back in the log for all transactions T that have written a `start_transaction` record into the log but have not written their commit record yet
- These transactions may have to be **rolled back** to undo their effect on the database during the recovery process
- Transactions that have written their commit record in the log must also have recorded all their `WRITE` operations in the log, so their effect on the database can be redone from the log records

PROPERTIES OF TRANSACTIONS: ACID

Atomicity

- A transaction is an atomic unit of processing
- It is either performed in its entirety or not performed at all

Consistency preservation

- A transaction is consistency preserving if its complete execution takes the database from one consistent state to another

PROPERTIES OF TRANSACTIONS: ACID

Isolation

- A transaction should appear as though it is being executed in isolation from other transactions
- The execution of a transaction should not be interfered with by any other transactions executing concurrently

Durability or permanency

- The changes applied to the database by a committed transaction must persist in the database
- These changes must not be lost because of any failure

TRANSACTION ROLLBACK

- If a transaction fails for whatever reason after updating the database, it may be necessary to roll back the transaction
- If any data item values have been changed by the transaction and written to the database, they must be restored to their previous values
- The undo-type log entries are used to restore the old values of data items that must be rolled back

TRANSACTION ROLLBACK

	A	B	C	D
	30	15	40	20
[start_transaction, T_3]				
[read_item, T_3 , C]				
[write_item, T_3 , B, 15, 12]		12		
[start_transaction, T_2]				
[read_item, T_2 , B]				
[write_item, T_2 , B, 12, 18]		18		
[start_transaction, T_1]				
[read_item, T_1 , A]				
[read_item, T_1 , D]				
[write_item, T_1 , D, 20, 25]				25
[read_item, T_2 , D]				
[write_item, T_2 , D, 25, 26]				26
[read_item, T_3 , A]				

← System crash

- **Cascading rollback**

- If a transaction T is rolled back, any transaction S that has read the value of some data item X written by T must also be rolled back
 - Once S is rolled back, any transaction R that has read the value of some data item Y written by S must also be rolled back
 - And so on...
-
- T_3 is rolled back because it did not reach its commit point
 - T_2 is rolled back because it reads the value of item B written by T_3

RECOVERY TECHNIQUES

[start_transaction, T_1]
[write_item, $T_1, D, 20$]
[commit, T_1]
[start_transaction, T_2]
[write_item, $T_2, B, 10$]
[write_item, $T_2, D, 25$]

← System crash

- The [write_item,...] operations of T_1 are redone
- T_2 log entries are ignored by the recovery process

CHECKPOINTS IN THE SYSTEM LOG

- A [checkpoint] record is written into the log periodically at that point when the system writes out to the database on disk all DBMS buffers that have been modified
- As a consequence of this, all transactions that have their [commit, T] entries in the log before a [checkpoint] entry do not need to have their WRITE operations redone in case of a system crash, since all their updates will be recorded in the database on disk during checkpointing

RECOVERY TECHNIQUES

[start_transaction, T_1]
[write_item, T_1 , D , 20]
[commit, T_1]
[checkpoint]
[start_transaction, T_4]
[write_item, T_4 , B , 15]
[write_item, T_4 , A , 20]
[commit, T_4]
[start_transaction, T_2]
[write_item, T_2 , B , 12]
[start_transaction, T_3]
[write_item, T_3 , A , 30]
[write_item, T_2 , D , 25]

← System crash

- T_2 and T_3 are ignored because they did not reach their commit points
- T_4 is redone because its commit point is after the last system checkpoint

TRANSACTION PROCESSING IN MYSQL

- When a session is started in MySQL, theAUTOCOMMIT system variable is normally turned on
- The following statement turns off autocommit

```
SET @@AUTOCOMMIT = 0;
```

- When autocommit must be turned on again, issue this statement

```
SET @@AUTOCOMMIT = 1;
```

TRANSACTION PROCESSING IN MYSQL

- The change can be undone with the SQL statement ROLLBACK or made permanent with the COMMIT statement

- COMMIT statements make the changes permanent

COMMIT;

- ROLLBACK statements undo the changes

ROLLBACK;

TRANSACTION PROCESSING IN MYSQL

- It is also possible to undo only a part of a current transaction by using savepoints

```
SAVEPOINT savepoint_name;
```

- To use savepoints, we must extend the ROLLBACK statement

```
ROLLBACK TO SAVEPOINT savepoint_name;
```

REFERENCES

- Elmasri, Ramez and Shamkant B. Navathe (2016), Fundamentals of Database Systems, 7th edition, Addison Wesley.
- Rick F.van der Lans (2007), SQL for MySQL Developers, Addison Wesley.
- Connolly, Thomas and Carolyn Begg (2015), Database Systems A Practical Approach to Design, Implementation, and Management, 6th edition, Pearson Education, ISBN 13: 978-1-292-06118-4, Essex, England.

NEXT WEEK: DATABASE SECURITY

- Introduction to Database Security Issues
- Concept of a View
- Discretionary Access Control
- Mandatory Access Control
- Adding & Removing Users
- Granting, Passing On, & Revoking Privileges
- Creating, Updating, & Deleting Views

```
SELECT *  
FROM WORLD  
WHERE SOMEONE  
LIKE '%YOU%';
```

```
EMPTY SET (0.00 SEC)
```

Vision

To become an **outstanding** undergraduate Computer Science program that produces **international-minded** graduates who are **competent** in software engineering and have **entrepreneurial spirit** and **noble character**.



Mission

1. To conduct studies with the best technology and curriculum, supported by professional lecturer
2. To conduct research in Informatics to promote science and technology
3. To deliver science-and-technology-based society services to implement science and technology